

Planning and Conformance:
Mission Geometry and the Projection Keystone

The Chatman Equation Thesis Program
(Praxis / Capability Physics)

Abstract

This paper consolidates two prior theses in this series – *Mission Geometry: Planning, Conformance, and Separability for Bounded Receipted Execution* and *The Projection Principle* – into one account, organized around a single keystone: the manufacturing morphism $\mathcal{A} = \mu(\mathcal{O}^*)$, its chained receipt, and the Comprehension– Verification cost-regime gap it makes possible, with PDDL planning, Petri-net conformance, POWL separability, and lexicographic cost arbitration stated as that keystone’s concrete geometric instantiation. Every claim is marked evidence-bound or speculative: evidence-bound claims cite the exact corpus RDF label and Lean 4 kernel receipt (bare Lean 4 core and, separately, a Mathlib-linked lane) that grounds them; speculative claims – a market-framed revenue-separation theorem, three planetary-scale engineering estimates, an interpretive claim about refusal-vocabulary legibility, and a self-audited-scale antibody clause – are quarantined into a single dedicated chapter, never mixed silently into the technical chapters. Three of the keystone’s own central theorems – Faithful Projection, the noncomprehension corollary, and Conservation of Consequence – are reported as stated but not machine-verified in either Lean lane on the receipts currently on disk, a limitation stated plainly rather than smoothed over.

Contents

1	Introduction	3
1.1	Thesis statement	3
1.2	What changes when two source theses are read as one	3
1.3	Structure of this document	4
2	The Projection Keystone: Admission, Manufacture, and Receipts	5
2.1	Rice quarantine and the admission map	5
2.2	The denial monoid	5
2.3	The Chatman equation and the manufacturing morphism	6
2.4	The law-bearing receipt	6
2.5	Faithful projection and its verification status	6
2.6	Conservation of consequence	7
3	Cost Regimes and the Comprehension–Verification Gap	8
3.1	Verification regimes	8
3.2	The Fleet Trust Budget	8
4	PDDL Action Law: Lifted Domains, Grounding, Bounded Branching	9
4.1	Lifted domains and grounding	9
4.2	Bounded branching	9
4.3	Relational membership via XOR filters	10
5	Numeric Fluents and the Attention Balance Law	11
5.1	The general attention calculus	11
5.2	The concrete PDDL fluent mechanics	11
6	The Marking Polytope: Petri Nets, State Equation, Cone Reachability	13
6.1	Nets, incidence, and the state equation	13
6.2	The reachability cone and its polytope relaxation	13
6.3	STRIPS grounding as a place/transition net	14
7	Conformance as Cone Membership: Farkas Separation Certificates	15
7.1	Conformance as polytope membership	15
7.2	Farkas’ lemma and the separating certificate	15
7.3	The certificate’s verification cost	16

8	Token-Replay Fitness as Normalized Distance	17
8.1	The replay verifier	17
8.2	Fitness and precision as population ratios	17
9	POWL 2.0: Recursive Partial Orders and Choice Graphs	19
9.1	The POWL grammar	19
9.2	Temporal plans as Hasse diagrams	19
10	Separability as a Process-Layer Rice Quarantine	20
10.1	The decomposition theorem	20
10.2	Separability as a decidable admission predicate	20
10.3	Normalization and language preservation	20
11	Cost Arbitration: Lexicographic Order and the Makespan Functional	22
11.1	The cost vector	22
11.2	Lexicographic order as an infinite-weight limit	22
11.3	The makespan functional	23
12	Instance: An Autonomous Build and the Trillion-Agent Status Byte	24
12.1	An unread interior, trusted by boundary receipts	24
12.2	The eight-bit status byte	24
12.3	Fleet arithmetic	24
12.4	Five named adversaries	25
13	Implementation Map	26
14	Appendix: The Datalog Index Layer	28
14.1	Nemo saturation	28
14.2	Symbol encoding	28
15	Speculative Extension: Planetary-Scale Projection	30
15.1	Maximum Reachable Revenue: a market-framed extrapolation	30
15.2	Planetary arithmetic beyond the exact count	31
15.3	A shared refusal vocabulary across agents	31
15.4	The self-audited-scale antibody clause	32
16	Limitations	33
17	Conclusion	35

Chapter 1

Introduction

1.1 Thesis statement

This paper consolidates two source theses into one account: *Mission Geometry: Planning, Conformance, and Separability for Bounded Receipted Execution* (`docs/thesis/03_planning_geometry.tex`), and *The Projection Principle: A First-Principles Theory of Bounded Receipted Execution for a Trillion Agents Beyond One Another's Comprehension* (`docs/thesis/projection_thesis.tex`). Read together, the two make one claim: the manufacturing morphism $\mathcal{A} = \mu(\mathcal{O}^*)$ and its chained receipt are a single formal keystone, and PDDL planning, Petri-net conformance, POWL separability, and lexicographic cost arbitration are its concrete geometric instantiation, not a separate theory bolted alongside it.

Every claim below traces to one of two artifact classes, cited by exact path at the point of use: (a) the corpus RDF at `docs/thesis/rdf/03_planning_geometry.ttl` and `docs/thesis/rdf/projection_thesis.ttl` whose `math:statement` literals are the source of every definition and theorem restated here; and (b) the Lean 4 kernel receipts at `tools/paper-factory/lean-pilot/formalization_receipts.jsonl` (bare Lean 4 core, no Mathlib) and `tools/paper-factory/lean-lake/mathlib_migration_receipts.jsonl` (the Mathlib-linked lane), whose `status` field is reported here exactly as it reads on disk, in both directions: neither upgraded past what the kernel accepted, nor downgraded past what it did.

`doc:register` marks every paragraph in this document EVIDENCE-BOUND or SPECULATIVE. EVIDENCE-BOUND means every quantitative or factual claim in the paragraph names the receipt line, corpus label, or source file that grounds it. Anything motivated by, but not derived from or checked against, an artifact is SPECULATIVE and is quarantined into Chapter 15, never mixed silently into a technical chapter – the same discipline `docs/thesis/math_manufacturing/thesis.tex` already applies to its own Chapter 5.

1.2 What changes when two source theses are read as one

The two source theses divide the same subject matter along an ownership line stated once, here, rather than re-argued in every chapter that could plausibly claim a topic. *The Projection Principle* owns the keystone proper: the admission monoid, the manufacturing morphism μ , the receipt chain, and the Comprehension–Verification cost-regime gap (Chapters 2–3 below), plus one chapter unique to it, the trillion-agent instance (Chapter 12). *Mission Geometry* owns the concrete planning and conformance mechanics: PDDL grounding, the Petri-net marking polytope, Farkas certificates, token-replay fitness, POWL separability, and cost arbitration (Chapters 4–11 below). Where both source outlines flag the same concept – attention-as-capacity, the Rice-quarantine rhetorical pattern,

the cost vector – this paper states the general form once, in the chapter that owns it, and the concrete form once, in the chapter that owns that, with an explicit cross-reference rather than a restatement.

Shared concept	General form owned by	Concrete instantiation owned by
Attention as bounded capacity	Ch. 5, §5.1 (<i>projection_thesis</i>)	Ch. 5, §5.2 (<i>planning_geometry</i>)
Rice-quarantine retraction pattern	Ch. 2, §2.1 (observation meanings)	Ch. 10 (workflow-net separability)
Cost vector / lexicographic order	Ch. 11, §11.2 (stated once, both sources verify it)	Ch. 11 (6-tuple <code>CostVector</code> operational form)
Marking polytope / Farkas / fitness / separability	Ch. 6–10 (<i>Mission Geometry</i> owns exposition)	<i>projection_thesis</i> cited only for the Rice-quarantine analogy tying separability to admission

Table 1.1: Ownership assignment for concepts flagged as shared by both source outlines. Stated once here; not re-argued per chapter.

Two corrections are carried forward from the source material rather than smoothed over. First, *projection_thesis*’s own formulation of the manufacturing morphism is adopted exclusively: μ is deterministic (M1) and bounded (M2) but need *not* be injective, so Conservation of Consequence (§2) is stated as *commitment*, not semantic recovery, when μ is not injective – fixing an earlier naive-uniqueness reading the source outline flags explicitly. Second, three of the keystone’s central theorems – Faithful Projection (`thm:faithful`), the noncomprehension corollary (`cor:noncomp`), and Conservation of Consequence (`thm:conservation`) – are, as of the receipts on disk, *not* machine-verified in either Lean lane: `thm:faithful` carries no bare-core pilot receipt line at all and is recorded **excluded** in the Mathlib-lane receipts; `cor:noncomp` and `thm:conservation` are recorded **blocked** with zero attempts in both lanes. This paper states all three as stated-but-not-machine-verified every place they are used, including inside Chapter 12’s instance argument, which depends on `thm:faithful`.

1.3 Structure of this document

Chapter 2 states the admission monoid, the Chatman equation, and the receipt chain. Chapter 3 states the verification-regime cost separation. Chapters 4–11 give the planning-and-conformance geometry: PDDL grounding and bounded branching, the attention balance law, the marking polytope, Farkas separation certificates, token-replay fitness, POWL separability, and lexicographic cost arbitration with the makespan functional. Chapter 12 reports the trillion-agent status-byte instance. Chapter 13 merges the two sources’ implementation-map tables into one. An appendix (Chapter 14) reports the Datalog index layer, demoted from a full chapter since it does not recur elsewhere in either source. Chapter 15 quarantines every claim either source itself flags as an estimate, a consequence-not-theorem, or a business-scale extrapolation, with one stated exception: `thm:mrr` is placed there by this paper’s own editorial judgment, not by any hedge *Mission Geometry* itself applies – the source states it as an unhedged theorem, verified in the bare-core Lean lane, alongside `thm:lex/cor:dominate/def:makespan` in Chapter 11’s own source material (§15 states the reclassification argument and its receipt history in full). Chapter 16 states what is not shown, without euphemism, and Chapter 17 closes.

Chapter 2

The Projection Keystone: Admission, Manufacture, and Receipts

2.1 Rice quarantine and the admission map

Axiom 2.1 (Observation space). There is a set $\mathcal{O}$, the observation space, whose elements are arbitrary finite records: logs, agent outputs, sensor frames, claims, tool responses. $\mathcal{O}$ carries no decidable semantics – “does this observation mean what it purports to mean” is not assumed computable.

Theorem 2.1 (Rice quarantine). *Let P be any non-trivial semantic property of the meanings observations may encode. Then $\{o \in \mathcal{O} : P(o)\}$ is undecidable.*

(docs/thesis/rdf/projection_thesis.ttl, labels ax:obs/ thm:rice.) Both are verified in the bare-core pilot lane and, since the migration to Mathlib, as a direct corollary of Mathlib’s own `ComputablePred.rice_2` and `ComputablePred.halting_problem` (`Mathlib.Computability.Halting`, formalizing Carneiro 2019) rather than a from-scratch hand proof – a citation of an existing published result, replacing 13 corpus-specific axioms in the bare-core file with zero.

Definition 2.1 (Admission map). Fix a decidable $\mathcal{O}^* \subseteq \mathcal{O}$ on which a finite battery of obligations $g_1, \dots, g_m : \mathcal{O} \rightarrow \{0, 1\}$ is computable. The admission map is the partial retraction $\alpha : \mathcal{O} \rightarrow \mathcal{O}^* \cup \{\perp\}$, $\alpha(o) = \rho(o) \in \mathcal{O}^*$ if $\bigwedge_i g_i(o) = 1$, else $\perp$, with ρ a computable bounding/normalization and $\perp$ the distinguished refusal. Admission is idempotent on its image: $\alpha \circ \alpha = \alpha$.

Rice quarantine (Theorem 2.1) is the reason admission cannot decide meaning in general; `def : adm` instead retracts onto a decidable sub-language, verified in both Lean lanes (docs/thesis/rdf/projection_thesis.ttl, `def : adm`).

2.2 The denial monoid

Construction 2.1 (Denial monoid). Let $D = (\{0, 1\}^n, \vee, \mathbf{0})$ be the commutative idempotent monoid of denial words over n independent obligation lanes, componentwise OR, identity $\mathbf{0}$. Each obligation g_i contributes a lane map $d_i : \mathcal{O} \rightarrow \{0, 1\}^n$ with $d_i(o) = \mathbf{0} \iff g_i(o) = 1$. The total denial is $d(o) = \bigvee_i d_i(o)$, and $\alpha(o) \neq \perp \iff d(o) = \mathbf{0}$.

Proposition 2.2 (Semilattice, monotone admission). *$(\{0, 1\}^n, \vee)$ is the join-semilattice of $2^{[n]}$ with bottom $\mathbf{0}$. Adding obligations only enlarges denials: for $G' \supseteq G$, $d_{G'}(o) \succeq d_G(o)$, so the admitted set shrinks monotonically under $G' \supseteq G$.*

Both verified in both Lean lanes (`con:denial/prop:semilattice, docs/thesis/rdf/projection.thesis.ttl`). Refusal is data – a bit vector, not a side effect – matching CLAUDE.md’s own invariant that every error is a typed `Refusal` variant.

One further claim about the denial monoid appears only in *projection-thesis*’s prose, in the source’s own words: “At a trillion agents this taxonomy is a shared vocabulary: a refusal category is legible to a recipient who never saw the sender’s interior.” The source outline itself marks this claim SPECULATIVE – it is a legibility/interpretive claim about a fixed enumeration of refusal categories, not a corollary the denial-monoid construction above proves. It is deferred to Chapter 15 and is not asserted here.

2.3 The Chatman equation and the manufacturing morphism

Definition 2.2 (Manufacturing morphism, the Chatman equation). $\mu : \mathcal{O}^* \rightarrow \mathcal{A}$ is a deterministic computable map subject to (M1) determinism/reproducibility – $\mu(x)$ depends only on x , equal admitted inputs give bit-identical artifacts – and (M2) boundedness – μ factors through a representation of size bounded by fixed structural constants, so μ terminates with a priori bounded cost. The Chatman equation is $\mathcal{A} = \mu(\mathcal{O}^*)$, operationally $a = \mu(\alpha(o))$ when $\alpha(o) \neq \perp$, and $\mu(\perp) = \perp$.

Verified in both Lean lanes (`def:mu, docs/thesis/rdf/projection.thesis.ttl`). Determinism is not injectivity: distinct admitted observations may manufacture an identical artifact, which is exactly why Conservation of Consequence (Theorem 2.5) must be stated in terms of commitment rather than recovery.

2.4 The law-bearing receipt

Definition 2.3 (Law-bearing receipt). Fix a collision-resistant hash $H : \{0, 1\}^* \rightarrow \{0, 1\}^{256}$ and write $\text{dg}(\cdot) = H(\cdot)$. For a manufacture with admitted observation $x = \alpha(o)$, obligation set G with denial word $d(o)$, transition identity τ , artifact $a = \mu(x)$, replay result φ , refusal reason r , and implementation version v , the frame is $\text{fr} = \langle \text{dg}(x), \text{dg}(G), d(o), \tau, \text{dg}(a), \varphi, r, v \rangle$, and the chained receipt advances by $h_+ = H(h_- \parallel \text{fr})$. The receipt projection is $\pi : \Sigma \rightarrow \mathcal{R}$, $\pi(\sigma) = (\text{verdict}, h_+, \varphi, r)$, a tuple of at most κ chunks.

Verified in both Lean lanes (`def:receipt`). A receipt is a chained commitment to named fields, never a bare byte-hash – the same discipline this project’s own receipt tooling implements outside this thesis series (`docs/thesis/02_receipt_cryptography.tex`).

2.5 Faithful projection and its verification status

Theorem 2.3 (Faithful projection). *The receipt is faithful to lawful consequence exactly to the extent the frame commits its fields: for any committed field $\phi \in \{\text{dg}(x), \text{dg}(G), d(o), \tau, \text{dg}(a), \varphi, r, v\}$, producing a trace σ' that differs in ϕ yet matches the claimed h_+ requires a collision of H . Fields not committed by the frame are not attested.*

This paper states Theorem 2.3 but does not claim it is machine-verified: `tools/paper-factory/lean-pilot/for` carries no receipt line for `thm:faithful` at all (it is excluded a priori from the bare-core run, since bare Lean 4 core has no library support for its cryptographic/probabilistic content), and `tools/paper-factory/lean-lake/mathlib_migration_receipts.jsonl` records its status as `excluded` as well, with zero attempts. It is reported here as *stated, not machine-verified* in either lane, not downgraded further nor upgraded to a proved status it does not have on disk.

Corollary 2.4 (Non-comprehension of boundary trust). *An accepting verifier’s predicate factors as $\text{Verify} = V \circ \pi$ for a fixed $V : \mathcal{R} \rightarrow \{0, 1\}$ of input size $\leq \kappa$. Trust in the accept is a function of the κ -sized projection and the soundness of V , not of any agent’s comprehension of σ .*

Recorded **blocked** with zero attempts in both Lean lanes (**cor:noncomp**). Stated, not machine-verified.

2.6 Conservation of consequence

Theorem 2.5 (Conservation of Consequence). *(i) Causation: every actuated artifact a is the image of at least one admitted observation, $a \in \text{im } \mu$, and no $a \notin \text{im } \mu$ is actuated. (ii) Position uniqueness: each actuation appends exactly one receipt-chain position; the map $\text{actuation} \mapsto h_+$ is injective up to hash collision. (iii) Observation recoverability / commitment: if μ is injective on $\mathcal{O}^*$, the artifact determines the admitted observation (recoverability); otherwise, when the frame commits $\text{dg}(x)$, the receipt position uniquely commits to $(\text{dg}(x), \text{dg}(G), \tau, \text{dg}(a))$ (commitment, not semantic recovery).*

Recorded **blocked** with zero attempts in both Lean lanes (**thm:conservation**). This is the corrected formulation (§1.1 above): clause (iii) is deliberately weaker than a naive uniqueness claim would be, precisely because μ is only known deterministic, not injective. Being unattempted in Lean is a scope gap, not a proof failure; it is reported as such, not conflated with a rejected kernel run.

Chapter 3

Cost Regimes and the Comprehension–Verification Gap

3.1 Verification regimes

Definition 3.1 (Verification regimes). For a trace of n committed frames: boundary projection inspection (any verifier, per message) costs $O(\kappa)$; single-frame recomputation (spot check) costs $O(1)$ per frame; checkpointed/indexed replay (sampled audit) costs $O(\log n)$ or a bounded path; full chain replay (full audit) costs $O(n)$.

Verified in both Lean lanes (`def:regimes`, `docs/thesis/rdf/projection_thesis.ttl`). This table is stated once, here; Chapters 7 and 8 cite it rather than restate it when discussing certificate- and replay-verification cost.

3.2 The Fleet Trust Budget

Theorem 3.1 (Comprehension–Verification Gap; Fleet Trust Budget). *Let $\text{Comp}(\sigma) = \Theta(\dim \Sigma)$ be the cost of comprehending the interior and $\text{Ver}_\partial(\sigma) = O(\kappa)$ the cost of boundary verification of $\pi(\sigma)$. Then $\text{Comp}(\sigma)/\text{Ver}_\partial(\sigma) = \Omega(\dim \Sigma/\kappa) \rightarrow \infty$ as $\dim \Sigma \rightarrow \infty$, while boundary verification stays constant. For a fleet, boundary cost is $O(M \cdot \kappa)$ for M messages, $O(N \cdot \kappa)$ per round, $O(N^2 \cdot \kappa)$ all-to-all – linear in κ , not interior dimension.*

Verified in both Lean lanes (`thm:gap`). This is *not* a claim that all mechanical audit is $O(1)$: full replay is honestly $O(n)$ (Definition 3.1); the constant-cost claim is confined to boundary inspection of an already-committed projection, a distinction *projection_thesis* itself insists on and this paper preserves.

Chapter 4

PDDL Action Law: Lifted Domains, Grounding, Bounded Branching

4.1 Lifted domains and grounding

Definition 4.1 (Lifted domain). A lifted domain is a tuple $\mathcal{D} = (\mathcal{T}, \mathcal{P}, \mathcal{F}, \mathcal{S})$: a finite type hierarchy $\mathcal{T}$, a finite set $\mathcal{P}$ of predicate symbols, a finite set $\mathcal{F}$ of numeric function symbols (fluents), and a finite set $\mathcal{S}$ of action schemas; a durative schema carries typed parameters, a duration constraint, a condition C_s , and an effect E_s .

Verified in both Lean lanes (`def:domain, docs/thesis/rdf/03_planning_geometry.ttl`).

Definition 4.2 (Grounding). Given a domain $\mathcal{D}$ and a problem $(\mathcal{O}, \mathbf{m}_0, \nu_0, \gamma)$, grounding substitutes every type-compatible object tuple into each schema’s parameters, producing a finite set of ground actions; a grounded state is a pair $(\mathbf{m}, \nu)$ of a set $\mathbf{m}$ of true ground atoms and a fluent valuation ν .

Verified in both Lean lanes (`def:ground`).

4.2 Bounded branching

Theorem 4.1 (Bounded branching). *Let $\mathcal{D}$ have schemas of maximum parameter arity k over $|\mathcal{O}| = N$ objects. The number of ground actions is at most $|\mathcal{S}| \cdot N^k$, a finite constant independent of the goal; consequently a forward search over grounded actions has a branching factor bounded a priori, and μ ’s manufacture terminates within a fixed depth bound.*

Verified in the bare-core Lean lane (`tools/paper-factory/lean-pilot/formalization_receipts.jsonl, thm:bounded-ground, 1 attempt`). In the Mathlib-linked lane it is recorded `unformalized` after 3 attempts, a regression against the bare-core proof: the final attempt fails on `Unknown constant `Finset.sum_le_sum`` and `Unknown identifier `smul_eq_mul`` against the current Mathlib snapshot (`tools/paper-factory/lean-lake/mathlib_migration_receipts.jsonl`) – a lookup failure against a moving library target, not a defect in the bare-core proof, which remains verified on its own terms.

In `bcinr-pddl` – a sibling git repository at `../bcinr`, outside praxis’s own `crates//src/trees`, reached only via the path dependency in `Cargo.toml`, not vendored into this codebase – the ground-action count is additionally hard-capped by `PDDL8_MAX_GROUND`; exceeding it is a typed `BoundExceeded` refusal rather than an exception or unbounded search, and its `GroundProblem::find_plan`

(`bcinr-pddl/src/ground.rs`) is breadth-first search over ground-atom sets whose own failure mode is a receipted refusal (`NoAdmittedPlan`), never a divergence. Praxis’s own in-repo `crates/pddl-index/src/ground.rs` implements the analogous but distinctly named `IndexedGroundProblem::find_plan`, over its own `GroundError` enum; the two are separate types in separate crates and this paragraph cites only the former, `bcinr-pddl`’s. This is a code-level remark, not a separate corpus statement, cited in the source outline against no additional Lean label.

4.3 Relational membership via XOR filters

Construction 4.1 (Relational membership via XOR filter). Let $R \subseteq \mathcal{O}$ be the reachability fixpoint fact set. A frozen fact store builds an 8-bit XOR filter over FNV-1a hashes of R , with index mapping via the fastrange reduction $\text{reduce}(x, n) = (x \times n) / 2^{32}$; membership is gated by $x \in B$ if $F(x) = \text{true}$, else false, with false-positive rate $\approx 0.4\%$ and false negatives 0.

Verified in both Lean lanes (`con:xorf`).

Chapter 5

Numeric Fluents and the Attention Balance Law

5.1 The general attention calculus

Definition 5.1 (Attention balance, general form). Let $c(t) \geq 0$ be capacity and admitted action j draw rate $r_j \geq 0$ on $[s_j, e_j]$. Free capacity is $f(t) = c(t) - \sum_{j:t \in [s_j, e_j]} r_j$. A schedule is feasible iff $f(t) \geq 0$ for all t .

Verified in the bare-core Lean lane (`docs/thesis/rdf/projection.thesis.ttl`, `def:attnbalance`); recorded `unformalized` in the Mathlib lane after a Mathlib big-operator syntax mismatch across 3 attempts (budget exhausted, per the source outline). This is the general statement of attention-as-bounded- capacity that §5.2 below instantiates concretely for a single PDDL fluent.

Proposition 5.1 (Attention conservation, general form). *Total attention consumed, $\int_0^T \sum_j r_j \mathbf{1}_{[s_j, e_j]}(t) dt = \sum_j r_j (e_j - s_j)$, is independent of schedule ordering. Reordering redistributes when attention is spent, not how much; the optimization target is the makespan functional $T[\cdot]$ (§11.3 below) under precedence and the pointwise capacity constraint.*

Verified in the bare-core Lean lane; recorded `blocked` in the Mathlib lane (`prop:attnconservation`).

5.2 The concrete PDDL fluent mechanics

Definition 5.2 (Fluent draw, PDDL instantiation). A durative action j draws on a fluent ϕ if its effect decreases ϕ by r_j at start and increases it by r_j at end for rate $r_j \geq 0$, holding r_j units over $[s_j, e_j]$; the free level of ϕ at time t is $f_\phi(t) = \nu_0(\phi) - \sum_{j:t \in [s_j, e_j]} r_j$.

Verified in both Lean lanes (`def:balance`, `docs/thesis/rdf/03_planning_geometry.ttl`). This is Definition 5.1 specialized to a single PDDL numeric fluent under the borrow/return action pattern.

Proposition 5.2 (Conservation of the borrowed fluent). *If every action drawing on ϕ both decrements it by r_j at start and increments it by r_j at end, the net effect of each completed action on ϕ is zero, and the terminal valuation equals the initial: $\nu_{\text{end}}(\phi) = \nu_0(\phi)$; scheduling redistributes when ϕ is held, never how much exists.*

Verified in both Lean lanes (`prop:conserve`) – attention is redistributed in time, never created or destroyed.

Proposition 5.3 (Feasibility as free-level nonnegativity). *A durative-action schedule is feasible with respect to ϕ iff $f_\phi(t) \geq 0$ for all t ; for the unit-rate attention fluent this says the number of concurrently in-flight capabilities never exceeds the capacity $\nu_0(\mathbf{attention})$.*

Verified in both Lean lanes (**prop:balance**) – exactly Definition 5.1’s general feasibility condition, restated as the PDDL fluent’s own guard.

Chapter 6

The Marking Polytope: Petri Nets, State Equation, Cone Reachability

6.1 Nets, incidence, and the state equation

Definition 6.1 (Net). A net has p places and a finite set T of transitions; a marking is a vector $\mathbf{m} \in \mathbb{Z}_{\geq 0}^p$; a transition t is a pair $(\mathbf{m}_t^-, \mathbf{m}_t^+)$ of preset and postset, enabled at $\mathbf{m}$ iff $\mathbf{m} \geq \mathbf{m}_t^-$ coordinatewise, firing to $\mathbf{m}' = \mathbf{m} - \mathbf{m}_t^- + \mathbf{m}_t^+$.

Verified in both Lean lanes (`def:net`, `docs/thesis/rdf/03_planning_geometry.ttl`).

Definition 6.2 (Incidence matrix, state equation). Let $\mathbf{N} \in \mathbb{Z}^{p \times |T|}$ have column $\delta_t = \mathbf{m}_t^+ - \mathbf{m}_t^-$ for each transition t ; a firing sequence with Parikh vector $\mathbf{x} \in \mathbb{Z}_{\geq 0}^{|T|}$ satisfies the state equation $\mathbf{m} = \mathbf{m}_0 + \mathbf{N}\mathbf{x}$.

Verified in both Lean lanes (`def:incidence`).

6.2 The reachability cone and its polytope relaxation

Proposition 6.1 (Cone necessity). *Every marking $\mathbf{m}$ reachable from $\mathbf{m}_0$ satisfies the state equation for some $\mathbf{x} \in \mathbb{Z}_{\geq 0}^{|T|}$, hence lies in the integer points of $\mathbf{m}_0 + \text{cone}(\mathbf{N})$ intersected with $\mathbb{Z}_{\geq 0}^p$; the state equation is necessary for reachability but not sufficient.*

Recorded `unformalized` in the bare-core Lean lane after 3 attempts: the induction step fails on an index-shift mismatch in `incidenceSumAux` and an `omega` failure relating a `Nat` truncated-subtraction expression to its `Int`-cast components (`tools/paper-factory/lean-pilot/formalization_receipts.jsonl`, `prop:cone`). Verified in the Mathlib-linked lane in 2 attempts (`tools/paper-factory/lean-lake/mathlib_migrat` depending only on Lean’s three standard foundational axioms. The insufficiency half (spurious solutions exist) is the classical Petri-net fact of Murata 1989, cited rather than reproved in either lane – an external-citation proof step, not an in-paper derivation.

Definition 6.3 (Marking polytope). The marking polytope of a net is the relaxation $\mathcal{P} = \{\mathbf{m}_0 + \mathbf{N}\mathbf{x} : \mathbf{x} \geq \mathbf{0}\} \cap \{\mathbf{m} \geq \mathbf{0}\} \subseteq \mathbb{R}_{\geq 0}^p$, the continuous translated cone whose integer points over-approximate the reachable set.

Recorded `blocked` (not attempted) in the bare-core lane, verified in the Mathlib-linked lane (`def:polytope`). *projection_thesis*’s own `def:reachcone` states the same reachable-set-in-a-cone content in different notation; this paper states it once, here, as `def:polytope`, and does not restate `def:reachcone` separately.

6.3 STRIPS grounding as a place/transition net

Construction 6.1 (STRIPS as a place/transition net). Take the places to be the ground atoms; a ground action t with delete-effects $\mathbf{m}_t^-$ and add-effects $\mathbf{m}_t^+$ is a transition, and `GroundProblem::find_plan` searches marking space by BFS with the successor relation exactly transition firing.

Proposition 6.2 (Safe-net bit identity). *On a safe (1-bounded) net, the integer firing rule $\mathbf{m}' = \mathbf{m} - \mathbf{m}_t^- + \mathbf{m}_t^+$ coincides bit-for-bit with the branchless bitset update $\mathbf{enabled_tokens} \leftarrow (\mathbf{enabled_tokens} \mathbin{\&\&} \neg \mathbf{m}_t^-) \mid \mathbf{m}_t^+$, and the enabling test coincides with the branchless subset check $(\mathbf{enabled} \mathbin{\&\&} \mathbf{m}_t^-) \oplus \mathbf{m}_t^- = 0$.*

Both verified in both Lean lanes (`con:strips`, `prop:safe`). `PowlReplayVerifier`'s branchless update is thus not an approximation of the Petri-net firing rule on safe nets – it is the same integer operation, proved coincident.

Chapter 7

Conformance as Cone Membership: Farkas Separation Certificates

7.1 Conformance as polytope membership

Definition 7.1 (Conformance). A trace with final marking $\mathbf{m}^\dagger$ and transition-count vector $\mathbf{x}$ conforms to a net with initial marking $\mathbf{m}_0$ if $\mathbf{m}^\dagger \in \mathcal{P}$ and the $\mathbf{x}$ witnessing it is realized by a genuine firing ordering; it fails membership if $\mathbf{m}^\dagger \notin \mathcal{P}$.

Recorded `blocked` (not attempted) in the bare-core Lean lane; verified in the Mathlib lane (`def:conf`). *projection_thesis*’s `prop:conformmembership` states the same polytope-membership content; it is not restated separately here.

7.2 Farkas’ lemma and the separating certificate

Theorem 7.1 (Farkas separation). Fix $\mathbf{b} = \mathbf{m}^\dagger - \mathbf{m}_0$. Exactly one holds: (a) there exists $\mathbf{x} \geq \mathbf{0}$ with $\mathbf{N}\mathbf{x} = \mathbf{b}$; or (b) there exists $\mathbf{y} \in \mathbb{R}^p$ with $\mathbf{N}^\top \mathbf{y} \leq \mathbf{0}$ and $\mathbf{y}^\top \mathbf{b} > 0$, a separating hyperplane certifying $\mathbf{m}^\dagger \notin \mathbf{m}_0 + \text{cone}(\mathbf{N})$, and if (b) holds no firing sequence reaches $\mathbf{m}^\dagger$.

Recorded `blocked` (not attempted) in the bare-core Lean lane. Recorded `unformalized` in the Mathlib lane after 1 attempt, with the exact missing lemma named: `PointedCone.FG.isClosed` does not exist in the current Mathlib snapshot. Mathlib’s topological Farkas machinery (`ProperCone.hyperplane_separation`, `Mathlib.Analysis.Convex.Cone.InnerDual`) would give the alternative once `cone(N)` is packaged as a `ProperCone`, but that packaging requires `IsClosed(cone(N))` – the topological half of the Minkowski–Weyl theorem (a finitely generated cone in a finite-dimensional real vector space is closed) – which the current Mathlib snapshot does not supply as a reusable lemma. This is reported `unformalized`, not faked with `sorry` or an axiom. The proof-in-paper itself leans on Farkas’ lemma as an external citation (Schrijver 1986), not an in-paper derivation, for its own first step.

projection_thesis’s own `prop:conformmembership` states materially the same claim – conformance is polytope membership, nonconformance yields a Farkas certificate – and is recorded `unformalized` in the Mathlib lane for a *different* reason: a `Matrix.dotProduct` identifier mismatch against Mathlib’s unqualified `dotProduct`, budget exhausted after 3 attempts. Both labels name the same mathematical content under two independent Lean files with two independent kernel-error causes; this paper cites `thm:farkas` as the canonical statement (owned by *Mission Geometry*) and records `prop:conformmembership`’s distinct failure mode here rather than silently merging the two into one reported status.

7.3 The certificate's verification cost

Corollary 7.2 (Nonconformance certificate). *A nonconformance certificate is a single vector $\mathbf{y} \in \mathbb{R}^p$ together with $\mathbf{N}^\top \mathbf{y} \leq \mathbf{0}$ and $\mathbf{y}^\top \mathbf{b} > 0$; checking it is a matrix product and two sign tests, independent of trace length, verifiable within a bounded context κ .*

Recorded **blocked** (not attempted) in both Lean lanes (**cor:cert**). Its cost bound, $O(p \cdot |T|)$, is exactly the boundary-verification regime of Definition 3.1: a nonconformance certificate is checkable at the same $O(\kappa)$ -shaped cost as any other receipted claim, independent of the trace's own length.

Chapter 8

Token-Replay Fitness as Normalized Distance

8.1 The replay verifier

Definition 8.1 (Replay verifier). These field names are quoted verbatim from the `PowlReplayVerifier` struct (`bcinr-powl-receipt/src/replay.rs:31--35`), an external crate reached only via the path dependency declared in `Cargo.toml` – a sibling git repository at `../bcinr`, outside praxis’s own `crates//src/` trees; praxis’s own `crates/praxis-core/src/replay_adapter.rs` only imports and wraps `bcinr_powl_receipt::replay::PowlReplayVerifier` and does not itself define the type. The verifier holds a marking $\mathbf{m}$ (`enabled_tokens`) initialized to the entry token, and accumulators `replayed`, `fitted`, `enabled_not_taken`; replaying a frame rejects an invalid node bit, requires $\mathbf{m} \geq \mathbf{m}^-$, records enabled-but-unconsumed tokens, fires $\mathbf{m} \leftarrow (\mathbf{m} \& \neg \mathbf{m}^-) \mid \mathbf{m}^+$, and sets the node bit in `replayed` and `fitted`.

Verified in both Lean lanes (`def:replay, docs/thesis/rdf/03_planning_geometry.ttl`).

Theorem 8.1 (Replay dichotomy). *Replaying a sequence of frames against the net either (i) completes with no violation, in which case it is a genuine firing sequence and $\varphi = 1$; or (ii) halts at the first frame whose preset is not contained in the current marking, returning a coordinate-localized witness of the earliest disabled transition.*

Verified in both Lean lanes (`thm:fitness`).

8.2 Fitness and precision as population ratios

Definition 8.2 (Fitness and precision). On finalization the verifier returns, in Q16.16 fixed point, $\varphi = |\text{fitted}|/|\text{replayed}|$ and $\text{precision} = |\text{replayed}|/(|\text{replayed}| + |\text{enabled_not_taken}|)$, where $|\cdot|$ is population count and $\varphi := 1$ when the denominator is zero.

Proposition 8.2 (Honest, orthogonal ratios). *φ and precision are ratios of bit-populations of disjointly-maintained bitsets; neither can exceed 1, and $\varphi = 1$ is attained exactly on completed replays; `enabled_not_taken` feeds precision, not fitness, so the two metrics measure orthogonal deviations and cannot be traded against each other.*

Both verified in both Lean lanes (`def:fitness, prop:honest`). *projection_thesis*’s `def:fitnessdistance/thm:u` restate the same fitness-as-normalized-distance content ($\varphi = 1$ – the fraction of tokens forced onto

unenabled transitions) and are verified in the bare-core lane, blocked in the Mathlib lane; this paper treats `def:fitness/thm:fitness` above as the canonical statement and does not duplicate the definition a second time.

Chapter 9

POWL 2.0: Recursive Partial Orders and Choice Graphs

9.1 The POWL grammar

Definition 9.1 (POWL model). A POWL model over an activity alphabet A is one of: an activity $a \in A$ (a leaf); a partial order $(\{M_1, \dots, M_k\}, \prec)$ executed by respecting $\prec$ and running incomparable children concurrently; or a choice graph $(\{M_1, \dots, M_k\}, E)$ combined by exclusive choice (and cyclic/loop edges, POWL 2.0), taking exactly one live branch.

Verified in both Lean lanes (`def:powl`, `docs/thesis/rdf/03_planning_geometry.ttl`).

Proposition 9.1 (Bounded local geometry). *In a POWL model, the token marking local to a node ranges over the tokens of that node’s immediate children: a node of arity k has a local marking space of dimension $O(k)$, independent of the total size of the model; a node’s replay is checked in a working set bounded by its arity.*

Verified in both Lean lanes (`prop:local`).

9.2 Temporal plans as Hasse diagrams

Construction 9.1 (Temporal plan to POWL tape). `temporal_plan_to_powl_tape` (`bcinr-pddl/src/powl_bridge`, a sibling git repository at `../bcinr`, outside praxis’s own `crates//src/` trees, reached only via the path dependency declared in `Cargo.toml` – not the in-repo `crates/powl2-decompose` crate, which has no tape/mask machinery) converts a `TemporalPlan` into a tape of `PowlOpSpecs` (struct also defined in that same external file): each step i becomes an activity with a `pred_mask` of steps $j < i$ finishing before i starts and a one-hot `succ_mask` (both fields on the tape type in `bcinr-powl/src/tape.rs`, same external repository); a second pass performs transitive reduction, clearing bits of predecessors’ own predecessors, leaving only direct predecessors.

Proposition 9.2 (Hasse diagram). *Let $\prec$ be the strict order “ j finishes before i starts” on plan steps. The initialized `pred_mask` of step i is $\{j : j \prec i\}$, and after transitive reduction it is the covering relation of $\prec$; the resulting dependency graph is a DAG, and two steps are schedulable concurrently iff $\prec$ -incomparable.*

Both verified in both Lean lanes (`con:tape`, `prop:hasse`).

Chapter 10

Separability as a Process-Layer Rice Quarantine

10.1 The decomposition theorem

Theorem 10.1 (Separable decomposition). *A sound, separable workflow net admits a recursive decomposition into a POWL model whose every internal node is a partial order or a choice graph and whose leaves are activities, with each node’s local marking geometry dimension bounded by its arity, independent of the global net size.*

Verified in both Lean lanes (`thm:sep, docs/thesis/rdf/03_planning_geometry.ttl`). The underlying decomposition result is attributed to Kourani and van der Aalst and cited, not reproved, in the source paper itself; the Lean formalization on record should be read as capturing the structural corollary this paper states – bounded local marking dimension under decomposition – not as an independent re-derivation of the cited external decomposition theorem. *projection_thesis* restates the same theorem as its own `thm:sep` (verified both lanes) and additionally frames separability as “a process-layer Rice quarantine”; that framing is the general Rice-quarantine pattern of §2.1, specialized to processes, and is stated here as the specialization, not re-derived.

10.2 Separability as a decidable admission predicate

Proposition 10.2 (Separability as a decidable admission predicate). *Let `sep` be the decidable predicate “the net is sound and separable.” It is a legitimate admission obligation: total and computable, retracting the space of arbitrary control-flow onto the decidable sub-language of POWL-expressible processes; a separable net is admitted, a non-separable net is refused with reason.*

Verified in both Lean lanes (`prop:quarantine`). This is the process-layer instance of Definition 2.1’s admission map, with `sep` as the obligation and the POWL decomposition as ρ .

10.3 Normalization and language preservation

Construction 10.1 (Sub-net normalization). Given a partition part $T' \subseteq T$ with entry places P_{in} and exit places P_{out} , the sub-net is normalized to a valid WF-net by adding fresh boundary places p_s, p_e and silent transitions $\tau_{\text{in}}, \tau_{\text{out}}$, redirecting boundary arcs onto a unified source and sink.

Theorem 10.3 (Language-preserving conversion). *Let N be a safe and sound workflow net, and let $\psi = \text{convert}(N)$ be its POWL 2.0 conversion. Then for any prefix trace length k , $\mathcal{L}_k(N) = \mathcal{L}_k(\psi) = \mathcal{L}_k(\text{recompose}(\psi))$.*

`con:normalize` is verified in both Lean lanes. `thm:lang-correct` carries no proof body at all in the source `.tex` – no `begin{proof}` block – and is reported here as asserted, not derived, in the source paper, even though the corpus records a Lean formalization for it in both lanes; what exactly that Lean statement encodes should be checked against `tools/paper-factory/lean-pilot/thm_lang_correct.lean` before citing it as a fully in-paper-proved result, a distinction this paper flags rather than smooths over.

Chapter 11

Cost Arbitration: Lexicographic Order and the Makespan Functional

11.1 The cost vector

Definition 11.1 (Cost vector). Assign a plan $\mathbf{c} = (c_0, c_1, \dots, c_k)$, with $c_0 \in \{0, 1\}$ the unadmitted indicator (0 = admitted) and $c_1, \dots, c_k$ bounded secondary costs, ordered lexicographically ($\preceq_{\text{lex}}$), cheapest-first. The router’s operational instance is the 6-tuple $\text{CostVector} = (c_0, \dots, c_5) = (\text{admitted}, \text{unreceipted_mutation_risk}, \text{human_attention_seconds}, \text{token_cost}, \text{latency_ms}, \text{context_switches})$ (field names verbatim from the `CostVector` struct at `capability_router.rs:107--122` in the `bcinr-pddl` crate – a sibling git repository at `../bcinr`, outside praxis’s own `crates/src/` trees, reached only via the path dependency declared in `Cargo.toml`; this and the other `bcinr-*` code loci cited in this chapter and Chapter 9 (`con:tape`) are not part of the reviewed praxis codebase itself), with the refused vector `refused()` = (unadmitted, 255, ∞ , ∞ , ∞ , 255) as the top element.

Both the general form (`def:costvector`, *projection.thesis*) and the 6-tuple operational form (`def:cost`, *Mission Geometry*) are verified in both Lean lanes. They are stated once here as one definition, general form first, operational instance second, rather than twice.

11.2 Lexicographic order as an infinite-weight limit

Theorem 11.1 (Lexicographic order as an infinite-weight limit). *Let secondary coordinates be bounded, $|c_i| \leq M$ for $i \geq 1$. Define $C_\lambda(\mathbf{c}) = \sum_{i=0}^k \lambda^{k-i} c_i$ with $\lambda > 1$. There is a threshold Λ such that for all $\lambda > \Lambda$, $\mathbf{c} \preceq_{\text{lex}} \mathbf{c}' \iff C_\lambda(\mathbf{c}) \leq C_\lambda(\mathbf{c}')$; as $\lambda \rightarrow \infty$ the weight on c_0 diverges relative to all others.*

Corollary 11.2 (Unlawfulness dominates). *An admitted plan sorts strictly before any refused plan regardless of its secondary costs: for any finite risk, makespan, token, latency, and switch values, an admitted `CostVector` is $\prec_{\text{lex}}$ the refused top element; the price of unlawfulness is infinite in the limit that defines the order.*

Both verified in both Lean lanes, in both source labelings (`thm:lex/cor:dominate`, *Mission Geometry*; `thm:lex`, *projection.thesis*). Routing determinism is receipted – identical tasks produce identical BLAKE3 route chains – a code-level remark citing the `routing_is_deterministic_same_task_same_route` test (`bcinr-pddl/src/capability_router.rs:340`, an admitted two-effect task asserting equal `route_chain` across two routing calls), with no separate corpus label.

11.3 The makespan functional

Definition 11.2 (Makespan functional). Given the precedence DAG with per-op durations d_i and release times, the forward pass computes $es_i = \max(r_i, \max_{j \prec i} ef_j)$, $ef_i = es_i + d_i$; the makespan is $T = \max_i ef_i$; the backward pass computes $ls_i = lf_i - d_i$; the slack of op i is $ls_i - es_i$, and the critical path is the set of zero-slack ops.

Proposition 11.3 (Longest path). *The makespan T equals the length of the longest ($\prec$ -)path in the precedence DAG weighted by durations, and an op has zero slack iff it lies on some longest path.*

Proposition 11.4 (Capacity sensitivity). *Let $T(\phi = c)$ be the makespan the planner finds when fluent ϕ 's initial capacity is c . The capacity delta reports $(T(c-1), T(c), T(c+1))$, and ϕ is binding iff $T(c-1) \neq T(c)$ (or infeasible at $c-1$); this is a finite-difference sensitivity of the specific schedule the greedy planner produced, not a dual shadow price of an optimal scheduler – the source paper explicitly disclaims the stronger reading.*

All three verified in both Lean lanes (`def:makespan`, `prop:critical`, `prop:sensitivity`).

Chapter 12

Instance: An Autonomous Build and the Trillion-Agent Status Byte

12.1 An unread interior, trusted by boundary receipts

An autonomous build with an interior of roughly 2.5×10^6 tokens, unread by any agent, is trusted entirely via boundary receipts: verdict, chain hash, fitness = 1.0, receipted refusal count, pass rate, and an oracle-agreement flag. This claim, as *projection.thesis* states it, cites Theorem 2.3 and Theorem 3.1 together with Proposition 2.2; it therefore inherits Theorem 2.3’s unresolved verification status (Ch. 2 above) exactly, and this paper states the inheritance rather than silently upgrading the build-trust claim past what `thm:faithful` itself has earned on disk.

12.2 The eight-bit status byte

Construction 12.1 (Eight-bit status byte). Project each agent to a status byte $b \in \{0,1\}^8$ whose lanes name admitted, evidence-ok, within-budget, authority-bound, healthy, conformant, receipted, replayable. A fleet of N agents is a bit-packed vector; a 64-bit word holds 8 agents; a fleet admission sweep is the branchless mask reduction of the denial construction (Construction 2.1) applied word-parallel.

Verified in both Lean lanes (`con:agent8`). Each bit is set only by a named mechanism, never by self-report – the worked numeric example in the source: a full status byte `0xFF` when every lane passes, dropping to `0x3F` with refusal category “Prerequisites” when ledger evidence is absent.

12.3 Fleet arithmetic

Proposition 12.1 (Fleet arithmetic, exact count). *For a fleet of N agents under Construction 12.1: the status vector occupies exactly N bytes, packed to $N/8$ 64-bit words; a full admission sweep is $N/8$ masked-OR word operations.*

This part of `prop:planetary` (part (a) in the source) is exact counting, verified in both Lean lanes, and is the only part of `prop:planetary` stated in this chapter. Parts (b)–(d) – the compressed-status estimate, the affordability-ratio estimate, and the consequence-not-theorem about planetary control – are engineering and order-of-magnitude estimates the source itself labels as such, and are quarantined into Chapter 15 rather than stated here alongside the exact count.

12.4 Five named adversaries

The source instance names five adversaries – forged self-report, broken replay, missing policy digest, non-separable workflow, unreceipted claim – each defeated by a specific mechanism rather than by inspecting the interior: forged self-report by Theorem 2.3 (stated, not machine-verified, Ch. 2), broken replay by Theorem 8.1 (verified, Ch. 8; the *projection_thesis* corpus additionally labels the same content `thm:unitfitness`), non-separable workflow by Theorem 10.1 (verified, Ch. 10), and unreceipted claim by `prop:selfauditedscale`, which is itself recorded `blocked` with zero attempts in both Lean lanes and is quarantined to Chapter 15 below as the “antibody clause.” This chapter’s adversary list therefore inherits a mixed evidentiary status across its five entries, stated per-entry rather than as one uniform “defeated” claim.

Chapter 13

Implementation Map

Both source theses carry their own implementation-map appendix, structurally identical in form (a table of formal object $\rightarrow$ code locus); consolidating them into two merged tables here replaces both, rather than adding a third alongside them.

Formal object	Code locus
Admission map α (Def. 2.1)	admission-obligation battery evaluated per manufacture request
Denial monoid (Con. 2.1)	bit-vector refusal reasons, componentwise OR
Manufacturing morphism μ (Def. 2.2)	<code>src/mfg.rs</code> (RDF $\rightarrow$ PDDL) and the Lean 4 autoformalization pipeline (Math $\rightarrow$ Lean), the two working μ -pipeline instances documented in <code>docs/thesis/math_manufacturing/</code>
Frame / receipt chain (Def. 2.3)	BLAKE3 chained receipt, genesis-folded
Projection π (Cor. 2.4)	bounded verdict/hash/fitness/reason tuple
Cost vector (Def. 11.1)	router <code>CostVector</code> , lexicographic comparator
Status byte (Con. 12.1)	per-agent 8-bit lane vector, word-parallel sweep

Table 13.1: Merged implementation map, keystone objects (both source appendices combined; not duplicated).

Formal object	Code locus
Grounding (Def. 4.2)	<code>bcinr-pddl</code> * grounding loop, <code>PDDL8_MAX_GROUND</code> bound
Forward search (Thm. 4.1)	<code>GroundProblem::find_plan</code> *, BFS over ground-atom sets
Marking / firing (Def. 6.1, Prop. 6.2)	<code>PowlReplayVerifier</code> * branchless bitset update
Farkas certificate (Thm. 7.1)	nonconformance witness $\mathbf{y}$, $O(p \cdot T)$ check
Fitness/precision (Def. 8.2)	Q16.16 population-count ratios, replay accumulators
Separable decomposition (Thm. 10.1)	POWL 2.0 conversion, <code>temporal_plan_to_powl_tape</code> *
Datalog saturation (App. 14)	<code>pddl-index</code> / Nemo reachability index

Table 13.2: Merged implementation map, planning/conformance geometry objects. *These four loci (`bcinr-pddl`’s `GroundProblem`, `bcinr-powl-receipt`’s `PowlReplayVerifier`, and `bcinr-pddl`’s `temporal_plan_to_powl_tape`) live in the sibling `bcinr` git repository, outside praxis’s own `crates//src/` trees, reached only via path dependencies in `Cargo.toml`; `pddl-index` (Datalog saturation row) is praxis’s own in-repo crate, and `PDDL8_MAX_GROUND` is a constant from the shared `wasm4pm-compat` dependency used by both `bcinr-pddl` and praxis’s own `pddl-index`, whose own `IndexedGroundProblem` type (distinct from `bcinr-pddl`’s `GroundProblem`) also enforces it.

Chapter 14

Appendix: The Datalog Index Layer

This appendix reports the Datalog index layer as an implementation-level addendum, not a full chapter of the main argument: unlike every other construct in Chapters 4–11, Datalog saturation and symbol-dictionary encoding do not recur in *projection.thesis* and have no cross-reference into the keystone chapters. It is retained because the source outline records it as evidence-bound corpus content, not because it is central to this paper’s argument.

14.1 Nemo saturation

Definition 14.1 (Datalog saturation). Let $(\mathcal{D}, \mathcal{O}|)$ be a grounded PDDL problem with initial atom set $\mathbf{m}_0$ and ground action set T . The Nemo Datalog program Π has one fact per atom in $\mathbf{m}_0$ and one rule per ground action; the saturation $\text{sat}(\Pi)$ is the least fixpoint $\text{lfp}(T_\Pi)$ of the immediate-consequence operator; an atom p is reachable iff $p \in \text{sat}(\Pi)$.

Proposition 14.1 (Saturation correctness). $p \in \text{sat}(\Pi)$ iff there exists a sequence of ground actions whose add-effects include p , i.e. p is reachable from $\mathbf{m}_0$ in the Petri-net sense.

Both verified in both Lean lanes (`def:saturation`, `prop:satcorrect`); the completeness half of `prop:satcorrect` cites the standard Datalog fixpoint theorem (Kowalski 1974, Lloyd 1987) as an external result, not an in-paper derivation.

14.2 Symbol encoding

Definition 14.2 (Symbol dictionary). A symbol identifier $\text{SymId} \in \mathbb{Z}_{>0}$ is a compact integer encoding of a ground atom or object name; this paper’s own cited corpus (`docs/thesis/rdf/03_planning_geometry.ttl`, `math:def.symdict`, matching `03_planning_geometry.tex`) states that `pddl-index` maintains a bidirectional map `SymDict: {strings}  $\leftrightarrow$   $\mathbb{Z}_{>0}$` , encoded forward as `FxHashMap<String, u32>` and reverse as `Vec<String>`; zero is reserved as undefined. The live `crates/pddl-index/src/dict.rs` disagrees with its own declared corpus on two points, undisclosed until now: the struct is named `Dict`, not `SymDict` (declared `crates/pddl-index/src/dict.rs:32`), and its forward map is a `std HashMap<String, u32>` (field `index`, `crates/pddl-index/src/dict.rs:36`), not `FxHashMap`. This paper states that discrepancy plainly, in the same spirit as its disclosures for `thm:mrr/thm:total` elsewhere, rather than silently substituting the live-code names for the corpus’s.

Verified in both Lean lanes (`def:symdict`).

Proposition 14.2 (Dictionary cost). *With symbol dictionary encoding, a ground atom $p(o_1, \dots, o_k)$ is a tuple of $k + 1$ `SymId` values; equality reduces to integer comparison, replacing $O(|T| \cdot k \cdot N)$ string comparisons with $O(1)$ -time integer counterparts.*

Verified in the bare-core lane; recorded **blocked** (not attempted) in the Mathlib lane (`prop:dictcost`).

Construction 14.1 (Typed relational join). Type-constrained grounding is a relational join: for a schema parameter v of type τ , the admissible objects are $\{o : \text{TypeIndex}::\text{satisfies}(o, \tau)\}$. This paper’s own cited corpus (`docs/thesis/rdf/03_planning_geometry.ttl`, `math:con_join`, matching `03_planning_geometry.tex`) states this as `TypeIndex::satisfies(SymId(o), SymId( $\tau$ ))`, operating on `SymId` values. The live method disagrees with that corpus, undisclosed until now: its real signature is `fn satisfies(&self, obj: &str, required: &str) -> bool` (`crates/pddl-index/src/ground.rs`: taking plain `&str` values, not `SymId`, and walking a string-keyed parent chain to depth bounded by the height of $\mathcal{T}$, reducing effective branching from N^k to a product of per-parameter type-compatible object counts. This paper states that discrepancy plainly, in the same spirit as its disclosures for `thm:mrr/thm:total` elsewhere, rather than silently substituting the live-code signature for the corpus’s.

Verified in both Lean lanes (`con:join`).

Chapter 15

Speculative Extension: Planetary-Scale Projection

Every claim in this chapter is motivated by, but not derived from or checked against, the evidence-bound chapters above, with one stated exception detailed below: `thm:mrr`’s underlying combinatorial-optimization statement was itself checked, and passed, in the bare-core Lean lane, and that identical statement already appears, undisclosed as speculative, in *Mission Geometry*’s own corpus RDF (§15 below states this in full – it is the same statement under one label, not a coincidental name collision between two different theorems). Each claim is wrapped in a dedicated `speculative` environment so a reader can mechanically distinguish it from Chapters 2–14. None of these claims should be read at the evidentiary standing of a theorem verified and placed by its own source in a technical chapter.

15.1 Maximum Reachable Revenue: a market-framed extrapolation

Speculation 15.1 (Maximum Reachable Revenue). *Let A be a set of independent client accounts, $\text{realized}(a)$ the revenue of account a under a target stage, and $\text{lawful_targets}(a)$ its evidence-gated target stages. The joint plan optimization decomposes linearly: $\max_{\text{joint plan}} \sum_{a \in A} \text{realized}(a) = \sum_{a \in A} \max_{t \in \text{lawful_targets}(a)} \text{realized}(a, t)$, reducing search complexity from $\Omega(\prod_a |T_a|)$ to $O(|A| \cdot |T_a|)$.*

This is a market/revenue-framed claim layered onto an otherwise mechanism-level chapter: “independent client accounts” and “realized revenue” are a business-scale extrapolation of a combinatorial-optimization fact (additive separability of a max over independent coordinates), not something evidenced by `praxis` code the way Chapter 11 and earlier are. Its placement here is this paper’s own editorial reclassification on business-framing grounds, not a hedge *Mission Geometry* itself applies: the source states `thm:mrr` as an unhedged theorem, formatted identically to its neighbors `thm:lex/cor:dominate/def:makespan`, immediately after `cor:dominate` in Chapter 11’s own source material (`docs/thesis/03_planning_geometry.tex`, lines 762–772). Like Theorem 10.3 above, `thm:mrr` carries no proof body at all in that source `.tex` – no `begin{proof}` block – per the corpus RDF’s own extraction notes (`docs/thesis/rdf/03_planning_geometry.ttl`, lines 21–26), a gap this paper discloses here on the same standard already applied to `thm:lang-correct`. `thm:mrr` is recorded `verified` in the bare-core Lean lane (3 attempts) and `unformalized` in the Mathlib lane after a `Finset.le_sup`’ motive-unification failure (3 attempts); this same statement – not a differently-scoped label that happens to share a name – already appears, undisclosed as

speculative, as an ordinary `dependsOn`-chained statement in *Mission Geometry*’s own corpus RDF (`docs/thesis/rdf/03_planning_geometry.ttl`, tagged fromPaper "03_planning_geometry" in `docs/thesis/rdf/corpus.ttl`), immediately following `cor:dominate/thm:lex` and depending on `def:cost` – the same technical cost-arbitration chain Chapter 11 above cites as evidence-bound. This paper’s own declared source corpus therefore does not itself separate this claim from its technical statement chain, in tension with §1.1’s promise that speculative material is never mixed silently into a technical chapter; this paper states that tension plainly here rather than reading it as a label collision between two unrelated theorems. A distinct label, `thm:mrrsep`, names a separate additive-separation statement and is recorded the other way around – `unformalized` in the bare-core lane (3 attempts, 4 kernel errors on the final attempt including two `omega` failures) and `verified` in the Mathlib lane (2 attempts). `thm:mrrsep` belongs to `docs/thesis/rdf/01_admission_algebra.ttl`, outside this paper’s own two-source scope (`03_planning_geometry.tex`, `projection_thesis.tex`), the same disclosure this paper makes for `thm:total` in Chapter 16; it is mentioned here only because its label sits adjacent to `thm:mrr` in the corpus’s Lean receipts, not because it is in scope. These are two distinct Lean files with two distinct, non-overlapping receipt histories; this paper does not conflate them under a single “MRR is verified” claim.

15.2 Planetary arithmetic beyond the exact count

Speculation 15.2 (Planetary arithmetic, estimates). *For a fleet of N agents (Construction 12.1): compressed status is ~ 10 GB and sweep time is $\Theta(N/(8B))$ at streaming bandwidth B – labeled by the source itself as an engineering estimate, not a theorem. The affordability ratio between one cache-line mask operation and one LLM admission decision is $\sim 10^7$ – an order-of-magnitude estimate, explicitly not a theorem. Consequence, not theorem: a comprehension-based or per-agent-LLM planetary control plane is infeasible by roughly $N \cdot 10^7$ against a feasible bit-parallel admission sweep; projection-based admission is the only route to planetary coherence the source argues for at $N = 10^{12}$ agents.*

The source paper’s own labeling – “engineering estimate,” “order-of-magnitude estimate,” “consequence, not theorem” – is kept verbatim here rather than upgraded to theorem language. The receipts for these sub-claims (`prop:planetary` parts (b)–(d)) are recorded `verified` in both Lean lanes, but that status attaches to the proposition *wrapper* the source paper itself states parts (b)/(c) are non-theorem estimates within – a Lean receipt on the wrapper does not convert an estimate into a machine-checked numeric claim, and this paper does not read it as doing so.

15.3 A shared refusal vocabulary across agents

Speculation 15.3 (A refusal vocabulary legible across agents). *Refusal categories, drawn from Construction 2.1’s n -lane denial monoid and organized under the source’s fixed eight-bucket surjection, are claimed, in the source’s own words, to form a vocabulary such that “a refusal category is legible to a recipient who never saw the sender’s interior” at trillion-agent scale.*

This is an interpretive/legibility claim about a fixed enumeration of refusal categories, not a corollary Construction 2.1 or Proposition 2.2 proves; those establish the algebraic structure of denial words, not that any particular 8-bucket enumeration is legible across arbitrary agents. It is recorded `verified` in both Lean lanes under `con:denial`’s own label, but that receipt attaches to the monoid construction, not to the legibility claim layered on top of it, which is why this paper states the legibility claim only here, quarantined, rather than in Chapter 2.

15.4 The self-audited-scale antibody clause

Speculation 15.4 (Self-audited-scale antibody clause). *Let a system emit claims c with receipts $r(c)$, and define the overclaim set $\text{Over} = \{c : \nexists r(c) \text{ with } V(\pi) = 1\}$. If the system enforces $\text{Over} = \emptyset$ (the *docs-exceed-mechanism* defect class), the admissible magnitude of a claim is bounded by the mechanism that receipts its limits, not by rhetoric.*

Recorded `blocked` with zero attempts in both Lean lanes (`prop:selfauditedscale`). It is stated here as a design aspiration this paper itself is trying to hold to – every claim above names its artifact or is marked speculative – not as a proved property of any running system.

Chapter 16

Limitations

This chapter states what is not shown, following `CLAUDE.md`'s reporting instruction to state findings, not verdicts, and without the quality adjectives the task does not ask for.

Three central keystone theorems – `thm:faithful`, `cor:noncomp`, `thm:conservation` – are stated in Chapter 2 but are not machine-verified in either Lean lane on the receipts currently on disk: `thm:faithful` has no bare-core receipt line and is recorded `excluded` in the Mathlib lane; the other two are recorded `blocked` with zero attempts in both lanes. Chapter 12's trillion-agent instance argument depends on `thm:faithful` and inherits this exact gap.

Farkas separation (`thm:farkas`) remains genuinely open in the Mathlib lane: the missing lemma is named precisely (`PointedCone.FG.isClosed`, the topological half of Minkowski–Weyl), not a vague timeout, and is recorded `blocked/not-attempted` in the bare-core lane entirely. Two related Petri-net propositions, `prop:cone` (bare-core) and `thm:bounded-ground` (Mathlib), each regressed against the other lane for reasons specific to that lane's own Mathlib-snapshot lookup failures (Ch. 4, Ch. 6 above) – these are lane-specific library gaps, not defects shared across both formalizations of the same statement.

The Mathlib-linked lane covers 178 of the corpus's 202 distinct labels (88%), not all of it, and not a claim of parity with the separately-verified bare-core lane's own scope; the two lanes are independently verified over overlapping but non-identical statement sets (`docs/thesis/math_manufacturing/rdf/thesis.ttl`, Ch. 4, reports this same 178/202 figure for the full corpus, of which this paper's own planning-geometry and projection-keystone labels are a subset). Three further Mathlib-lane failures exist corpus-wide beyond `thm:farkas` and `thm:bounded-ground` and fall within this document's own two-source scope: `def:attnbalance`, `prop:conformembership`, and `thm:mrr` – each has a specific, named kernel error on file (a library-lookup or motive-unification gap), not a generic failure; none is silently retried past the fixed 3-attempt bound. A fourth label, `thm:total`, is reported `unformalized` in the same Mathlib-migration receipts file but is not the same kind of gap: its recorded `last_error` (`mathlib_migration_receipts.jsonl:199`) states plainly that the theorem's own claim – `Category.Reserved` has empty preimage under `RefusalScenario.category` – is false against the already-verified `def:tax`, which maps three `RefusalScenario` variants (`logicContradiction`, `andonPull`, `andonEscalation`) to `Category.Reserved` (`Praxis/Corpus/def.tax.lean:67--69`, `Praxis/Corpus/thm_total.lean:19`); no amount of tactic work closes that goal because the goal itself is false against a dependency already on the kernel's own record, not merely difficult. This is a demonstrated falsification, not a tooling gap, and `thm:total` belongs to `docs/thesis/rdf/01_admission_algebra` outside this paper's own two-source scope (`03_planning_geometry.tex`, `projection_thesis.tex`). Unresolved and stated here rather than smoothed over: the bare-core lane (`tools/paper-factory/lean-pilot/form`) records this same label `thm:total` as `verified` (3 attempts, no error) against the identical falsified

claim – an unremarked cross-lane discrepancy of the same kind this paper flags elsewhere for `thm:mrr/thm:mrrsep` (Chapter 15) and `thm:farkas/prop:conformembership` (Chapter 7); this paper does not resolve that discrepancy, only reports it.

This is a single-kernel result: only Lean 4’s own kernel accepted or rejected each statement; no independent proof assistant cross-checks any of the `verified` dispositions reported in this paper. `thm:lang-correct` (Ch. 10) is flagged separately: its source `.tex` carries no proof body, so its status here is asserted-with-a-Lean-formalization-on-record, a weaker claim than proved-in-paper, stated as such rather than smoothed into an unqualified “proved.”

The specific rhetorical *device* of *projection_thesis*’s own front matter – addressed to 10^{12} concurrent agents, a bounded-context-capacity motif citing Miller and Cowan – is deliberately not carried forward as technical content anywhere in this paper outside the explicit disclaimer of Chapter 15: the source outline itself marks it speculative and instructs against duplicating it as if it were a technical claim, and this paper follows that instruction rather than restating the numeric/motif device a third time across the thesis series. This is narrower than the chapter’s own *naming* practice: Chapter 12’s title (“Instance: An Autonomous Build and the Trillion-Agent Status Byte”) and this paper’s own introductory structure overview (Chapter 1) and the paragraph immediately above in this same chapter all use “trillion-agent” as the name of that chapter’s status-byte instance, not as a restatement of the 10^{12} /Miller–Cowan motif; that naming usage is excepted from this paragraph’s claim and is not itself disclosed as speculative, since it names an evidence-bound instance argument (Chapter 12), not the front-matter framing device.

Chapter 17

Conclusion

This paper states one evidence-bound keystone (Chapter 2) and its concrete planning-and-conformance geometry (Chapters 4–11), one evidence-bound instance at declared scope (Chapter 12), and a clearly quarantined set of estimates and extrapolations (Chapter 15) that this paper does not claim are anything more than what their own source labels them. The correct adversarial question after this consolidation is not whether gaps exist – Chapter 16 names them plainly – but which specific admitted statement, blocked dependency, or evidence-bound claim above is wrong, checkable against the two receipts files and the corpus RDF that produced it, both cited by exact path throughout.

Bibliography

- [1] Alexander Schrijver. *Theory of Linear and Integer Programming*. Wiley, 1986.
- [2] Tadao Murata. Petri Nets: Properties, Analysis and Applications. *Proceedings of the IEEE*, 77(4), 1989.
- [3] Wil M. P. van der Aalst. *Process Mining: Data Science in Action*. Springer, 2016.
- [4] Humam Kourani and Wil M. P. van der Aalst. POWL: Partially Ordered Workflow Language. *International Conference on Business Process Management (BPM)*, 2023.
- [5] Robert Kowalski. Predicate Logic as Programming Language. *IFIP Congress*, 1974.
- [6] John W. Lloyd. *Foundations of Logic Programming*. Springer, 1987.
- [7] Mario Carneiro. Formalizing Computability Theory via Partial Recursive Functions. *International Conference on Interactive Theorem Proving (ITP)*, 2019.
- [8] Henry Gordon Rice. Classes of Recursively Enumerable Sets and Their Decision Problems. *Transactions of the American Mathematical Society*, 74(2), 1953.